

AI Sentiment Analysis Web Application using IBM Watson NLP and Flask



Nikolaos Belimpasakis
MEng in Computer Engineering and Informatics

Components

1. Overview and Objectives	2
2. Development Environment Setup	3
3. Sentiment Analysis Service Integration	4
4. Packaging the Sentiment Analysis Module	6
5. Unit Testing and Validation	7
6. Deployment of the Sentiment Analysis Application Using Flask	9
7. Static Code Analysis and Code Quality Assessment	14
8. Conclusion	15
9. References	16

1. Overview and Objectives

This project was developed as part of the IBM Generative AI Engineering [1] course and aims to provide hands-on experience in the design, implementation, testing, and deployment of an Artificial Intelligence-powered web application. The project combines concepts from Natural Language Processing (NLP), web development, software testing, and application deployment, allowing learners to gain practical experience with modern AI development workflows.

The main objective of the project is to build a web application capable of performing sentiment analysis [2] on textual input. Sentiment analysis is a Natural Language Processing technique that examines a piece of text and determines the emotional tone expressed within it, such as positive, negative, or neutral sentiment. This type of analysis is widely used in applications involving customer feedback, product reviews, social media monitoring, and opinion mining.

To implement the sentiment analysis functionality, the project utilizes IBM Watson NLP [3], a collection of Natural Language Processing libraries and services developed by IBM. Watson NLP provides pre-trained language models and analytical capabilities that enable developers to extract meaningful information from text without the need to train complex machine learning models from scratch. Through the integration of these tools, the application is able to analyze user-provided text and generate sentiment predictions efficiently.

In addition to the AI component, the project employs Flask [4], a lightweight Python web framework used for developing web applications and APIs. Flask provides a simple and flexible environment for handling user requests, processing input data, and displaying results through a web interface. By integrating IBM Watson NLP with Flask, the project demonstrates how AI services can be incorporated into a functional web application that can be accessed through a browser.

From an educational perspective, the project is designed to familiarize learners with the complete lifecycle of an AI-powered application. Beyond implementing the sentiment analysis functionality, the project also covers application packaging, testing, error handling, code quality assessment, and deployment. As a result, participants gain practical exposure not only to AI technologies but also to essential software engineering practices required for building reliable and maintainable applications.

2. Development Environment Setup

Before implementing the sentiment analysis functionality, it was necessary to prepare and verify the development environment that would be used throughout the project. All development activities were performed within the Skills Network Theia Lab [5], a cloud-based Integrated Development Environment (IDE) provided as part of the IBM Generative AI Engineering course. This environment offers a preconfigured workspace specifically designed for the laboratory exercises, ensuring compatibility with the required software tools and services.

The setup process began with the creation of a dedicated project workspace named `practice_project`. This workspace served as the primary location for storing and managing all project files. Once the workspace had been created, the project repository [6] was downloaded from GitHub using the Git version control system. Specifically, the repository was cloned through the `git clone` command, which copied the complete project structure into the development environment. Following the cloning process, the terminal session was redirected to the project directory using the `cd` command, allowing all subsequent development tasks to be executed within the project workspace.

After obtaining the project files, the development environment was validated to ensure that all required software components were available. The first step involved verifying the installed Python version using the `python3.11 -V` command. This verification was important because the laboratory instructions specified Python 3.11 as the target runtime environment, and using the correct version helps prevent compatibility issues during development and execution.

Once the Python installation had been confirmed, the availability of the required project dependencies was examined. This verification was performed using the `pip3.11 show` command, which provides detailed information about installed Python packages. Three essential libraries were inspected during this stage: Flask, Requests, and Pylint. Flask serves as the web framework responsible for handling user requests and rendering the application's web interface. Requests provides functionality for HTTP communication and data exchange, while Pylint is used as a static code analysis tool to evaluate code quality and identify potential issues. Verifying the installation of these libraries ensured that all required dependencies were available before the implementation phase began.

Following the dependency verification process, the contents of the project repository were inspected using terminal commands to confirm that all required files and directories had been downloaded successfully. The repository contained the main application file (`server.py`), project documentation files, and dedicated folders such as `templates` and `static`, which are commonly used in Flask

applications for storing HTML templates and static resources respectively. Examining the project structure at this stage provided an initial understanding of the application's organization and confirmed that the development environment had been configured correctly.

By completing these setup and verification procedures, a stable and fully functional development environment was established. This environment provided the foundation required for integrating IBM Watson NLP capabilities and implementing the sentiment analysis web application in the subsequent stages of the project.

3. Sentiment Analysis Service Integration

The core functionality of the application was implemented through the integration of the IBM Watson Natural Language Processing (NLP) service. Rather than executing a sentiment analysis model locally, the application communicates with a pre-deployed Watson NLP service hosted within the Skills Network laboratory environment. This approach allows the application to leverage IBM's pre-trained BERT-based sentiment analysis model [7] without requiring local model installation or training.

The implementation began with the creation of a dedicated Python module named `sentiment_analysis.py`. Within this module, a function called `sentiment_analyzer()` was developed to act as an intermediary between the application and the Watson NLP service. The function accepts a text string as input and prepares the data required by the sentiment analysis API.

To submit a sentiment analysis request, the application constructs a JSON payload containing the text to be analyzed. In addition, a custom HTTP header is included in the request to specify the identifier of the sentiment analysis model that should process the input. Once the payload and header information are prepared, the request is transmitted to the Watson NLP endpoint using the Python Requests library and the HTTP POST method.

Upon receiving the request, the Watson NLP service executes a BERT-based sentiment analysis workflow and returns a response containing multiple pieces of information related to the analyzed text. The response includes metadata, confidence scores, sentiment probabilities, and the overall sentiment classification generated by the model.

```
>>> sentiment_analyzer("I love this new technology")
{'documentSentiment':{'score':0.996954, 'label':"SENT_POSITIVE", 'mixed':false, 'sentimentMentions':[{span:{begin:0, end:26, text:"I love this new technology"}, "sentimentprob":{"positive":0.9941229, "neutral":0.0028645627, "negative":0.0030124863}}]}, "targetedSentiments":{"targetedSentiments":{}, "producerId":{"name":"Aggregated Sentiment Workflow", "version":"0.0.1"}}, "producerId":{"name":"Aggregated Sentiment Workflow", "version":"0.0.1"}}
```

Initially, the service response was returned directly as text. However, since the response follows the JSON format, additional processing was required to extract only the information relevant to the application. For this reason, the built-in Python `json` library was utilized to convert the textual response into a Python dictionary structure. This conversion enabled direct access to individual response elements through their corresponding keys.

Following the conversion process, the implementation extracted the two values most relevant to the sentiment analysis task: the sentiment label and the associated confidence score. These values are stored within the `documentSentiment` section of the Watson response and provide both the predicted sentiment category and the confidence level assigned by the model.

To simplify the interaction between the sentiment analysis module and the remainder of the application, the extracted information was returned as a compact Python dictionary containing only the `label` and `score` fields. This design choice reduced the complexity of the output while preserving the information required for subsequent processing and presentation within the web application.

The implementation of the `sentiment_analysis.py` module is shown below:

```
1  import requests
2  import json
3
4  def sentiment_analyzer(text_to_analyse):
5
6      url = "https://sn-watson-sentiment-bert.labs.skills.network/v1/watson.runtime.nlp.v1/NlpService/SentimentPredict"
7      myobj = {"raw_document": {"text": text_to_analyse}}
8      header = {"grpc-metadata-mm-model-id": "sentiment_aggregated-bert-workflow_lang_multi_stock"}
9
10     response = requests.post(url, json=myobj, headers=header)
11     formatted_response = json.loads(response.text)
12
13     return {
14         "label": formatted_response["documentSentiment"]["label"],
15         "score": formatted_response["documentSentiment"]["score"]
16     }
```

4. Packaging the Sentiment Analysis Module

After implementing the sentiment analysis functionality, the application code was reorganized into a dedicated Python package to improve modularity and maintainability. The package was named `SentimentAnalysis` and was created within the project directory.

The previously developed `sentiment_analysis.py` module was moved into the newly created package folder. In addition, an `__init__.py` file was added to the package directory. The purpose of this file is to indicate that the folder should be treated as a Python package and to enable the import of its modules from other parts of the application.

The `__init__.py` file was configured to reference the `sentiment_analysis` module, allowing the package contents to be accessed through standard Python import statements. As a result, the sentiment analysis functionality became available through the package structure rather than as a standalone module.

Following the packaging process, the implementation was validated by importing the `sentiment_analyzer` function from the newly created package and executing a test sentiment analysis request. The successful execution of the function confirmed that the package had been configured correctly and was ready to be integrated into the remaining components of the web application.

The final package structure consisted of the `SentimentAnalysis` directory containing the `sentiment_analysis.py` module and the corresponding `__init__.py` file, providing a cleaner and more scalable organization of the application code.

5. Unit Testing and Validation

Unit testing is a software testing technique in which individual units of code, such as functions or methods, are tested independently to verify that they behave as expected [8]. The primary objective of unit testing is to detect defects at an early stage of development and ensure that each component of the application produces correct and consistent results. By validating individual software units in isolation, developers can improve code reliability, simplify debugging, and facilitate future modifications without introducing unexpected errors.

In Python, unit testing is commonly performed using the built-in `unittest` framework, which provides mechanisms for defining test cases, executing them automatically, and comparing actual outputs against expected results [9]. This approach enables systematic verification of application functionality and contributes to the overall quality and maintainability of the software system.

To verify the correctness of the sentiment analysis functionality, a dedicated unit testing module named `test_sentiment_analysis.py` was developed. The testing process was implemented using Python's built-in `unittest` framework.

The testing module imports the `sentiment_analyzer()` function from the `SentimentAnalysis` package and evaluates its output against a set of known sentiment classifications. Three representative test cases were selected to cover positive, negative, and neutral sentiment scenarios. For each test case, the sentiment label returned by the Watson NLP service was compared with the expected label using assertion statements.

The objective of these tests was to ensure that the sentiment analysis module correctly identifies the sentiment category associated with a given textual input. Successful execution of all test cases provided evidence that the implemented functionality behaved as expected and that the integration with the Watson NLP service produced valid results.

The following figure presents the source code of the `test_sentiment_analysis.py` file:

```
1  from SentimentAnalysis.sentiment_analysis import sentiment_analyzer
2  import unittest
3
4
5  class TestSentimentAnalyzer(unittest.TestCase):
6
7      def test_sentiment_analyzer(self):
8
9          result_1 = sentiment_analyzer("I love working with Python")
10         self.assertEqual(result_1["label"], "SENT_POSITIVE")
11
12         result_2 = sentiment_analyzer("I hate working with Python")
13         self.assertEqual(result_2["label"], "SENT_NEGATIVE")
14
15         result_3 = sentiment_analyzer("I am neutral on Python")
16         self.assertEqual(result_3["label"], "SENT_NEUTRAL")
17
18
19  unittest.main()
```

After the successful execution of the unit tests, the output shown in the following figure was obtained.

```
theia@theia-abhishekg1:/home/project/practice_project$ python3.11 test_sentiment_analysis.py
.
-----
Ran 1 test in 0.501s

OK
```

6. Deployment of the Sentiment Analysis Application Using Flask

Following the implementation of the sentiment analysis functionality, the formatting of the API responses, and the packaging of the application into a reusable Python package, the final stage of the project involved deploying the system as a web application. This deployment was achieved using the Flask framework, which provides a lightweight mechanism for developing web services and connecting back-end processing logic with a user-facing interface.

The objective of this phase was to allow users to interact with the sentiment analysis system through a web browser rather than directly through the Python interpreter. To achieve this, the application was structured according to Flask's standard project architecture, separating the presentation layer, client-side scripting, and server-side processing components.

Application Structure

Before deploying the application, the project directory was organized into a structure compatible with Flask. The previously developed sentiment analysis module was placed inside the **SentimentAnalysis** package, while the user interface and client-side scripts were stored in dedicated directories.

The resulting project structure is presented in Figure X.

```
practice_project/  
├── SentimentAnalysis/  
│   ├── __init__.py  
│   └── sentiment_analysis.py  
├── templates/  
│   └── index.html  
├── static/  
│   └── mywebscript.js  
└── server.py
```

The **SentimentAnalysis** package contains the implementation of the sentiment analysis functionality, whereas the **templates** directory stores the HTML interface and the **static** directory contains the JavaScript code responsible for client-server

communication. The **server.py** file acts as the entry point of the web application and coordinates all interactions between the user interface and the sentiment analysis service.

Integration of the Sentiment Analysis Package

The first step in the deployment process consisted of integrating the previously developed package into the Flask application. To accomplish this, the required Flask components and the `sentiment_analyzer()` function were imported into the `server.py` file.

The imported Flask components provide the functionality required to handle incoming HTTP requests and render HTML pages, while the imported sentiment analysis function enables access to the Watson NLP service developed in the previous stages of the project.

Flask Application Initialization

After importing the required modules, a Flask application instance was created. This object serves as the central component of the web application and is responsible for managing all incoming requests and routing them to the appropriate processing functions.

The application was initialized with the name **Sentiment Analyzer**, providing a unique identifier for the Flask instance.

Development of the Sentiment Analysis Endpoint

A dedicated route named `/sentimentAnalyzer` was implemented to process user requests. This endpoint acts as the communication bridge between the web interface and the sentiment analysis package.

When a user submits text through the graphical interface, the text is transmitted to the Flask server through an HTTP GET request. The endpoint retrieves the submitted text, stores it in a local variable, and forwards it to the `sentiment_analyzer()` function. The returned dictionary contains the sentiment label and confidence score generated by the Watson NLP model.

The endpoint subsequently extracts these values and formats them into a human-readable sentence before sending the result back to the web interface.

Rendering the Web Interface

In addition to the sentiment analysis endpoint, a second route was implemented to serve the application's graphical user interface.

This route is associated with the root URL (/) and is responsible for rendering the **index.html** file stored inside the **templates** directory. Whenever a user accesses the application through a browser, Flask automatically invokes this route and returns the HTML page that constitutes the front-end interface.

The implementation of the page-rendering functionality is shown in Figure X.

Front-End and Back-End Communication

Communication between the browser and the Flask server is performed through the **mywebscript.js** file located in the **static** directory.

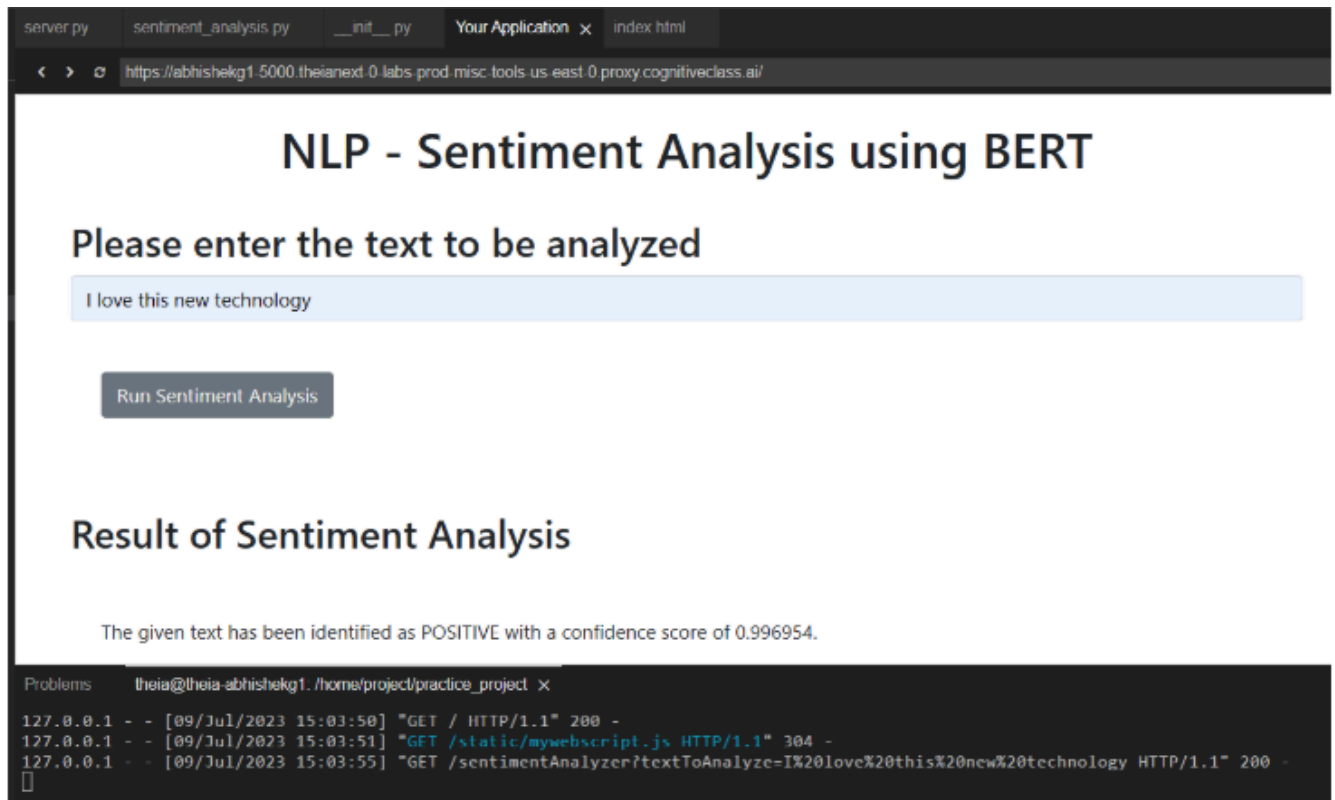
When the user presses the **Run Sentiment Analysis** button, the JavaScript code retrieves the text entered in the input field and constructs an HTTP request directed to the **/sentimentAnalyzer** endpoint. The submitted text is transmitted as a request parameter and processed by the Flask application.

After receiving the server response, the JavaScript code dynamically updates the web page and displays the sentiment analysis result without requiring a page refresh. This approach creates a more responsive and interactive user experience while maintaining a clear separation between presentation and processing logic.

Application Execution

After completing the implementation of all required routes, the Flask application was configured to execute on port **5000**. The server was launched through the Python interpreter by executing the **server.py** file.

During startup, Flask initializes the application instance, registers all routes, and begins listening for incoming requests. Once execution is successful, the application becomes accessible through a web browser, allowing users to submit text for sentiment analysis through the graphical interface.



Overall Application Workflow

The final application operates according to the following sequence of actions:

1. The user enters text through the HTML interface.
2. The JavaScript code captures the input and sends it to the Flask server.
3. The `/sentimentAnalyzer` endpoint receives the request.
4. The submitted text is passed to the `sentiment_analyzer()` function.
5. The Watson NLP service performs sentiment classification.
6. The sentiment label and confidence score are returned to the Flask server.
7. The server formats the response into a readable message.
8. The JavaScript code receives the result and displays it within the web interface.

Through this architecture, the sentiment analysis functionality developed in the previous stages is transformed into a complete web application that can be accessed and used through a standard web browser.

Important note: All source code files developed during the project have been successfully completed and uploaded to the repository. During local testing, it was observed that the application could not fully execute outside the IBM Skills Network environment. This limitation is related to the restricted accessibility of the Watson NLP endpoint used by the application, which appears to be available only within the laboratory infrastructure. Consequently, while the application structure and implementation are correct, full functionality depends on the IBM Skills Network environment where the sentiment analysis service is hosted.

7. Static Code Analysis and Code Quality Assessment

After completing the implementation, testing, packaging, and deployment phases of the application, a final code quality assessment was performed. The objective of this stage was to evaluate the source code against the recommendations of the Python Enhancement Proposal 8 (PEP 8), which defines the official style guidelines for Python programming.

Static code analysis was conducted using the **Pylint** tool, a widely used static analysis framework capable of identifying programming errors, code smells, style violations, and potential maintainability issues without executing the program itself. By analyzing the source code statically, developers can detect inconsistencies and improve overall code quality before deployment.

Static Code Analysis

The static analysis process focused on the `server.py` file, which contains the core Flask application responsible for handling requests and coordinating communication between the user interface and the sentiment analysis module.

Since Pylint had already been installed during the environment setup phase, no additional configuration steps were required. The analysis was executed directly from the terminal using the following command:

```
pylint server.py
```

The execution of this command triggered an automated inspection of the source code according to PEP 8 conventions and Pylint's built-in quality rules.

Analysis Results

Upon completion of the analysis, Pylint generated a score representing the overall quality and compliance of the code. The resulting score was **10.00/10**, indicating full compliance with the evaluated coding standards and the absence of significant style or quality issues.

The result obtained from the static code analysis is shown below:

```
theia@theia-abhishekg1:/home/project/practice_project$ pylint server.py
-----
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

The achieved score demonstrates that the developed application follows recommended Python coding practices and maintains a consistent and readable code structure. This contributes to improved maintainability, easier debugging, and better long-term software quality.

8. Conclusion

This project demonstrated the complete development lifecycle of an AI-powered web application for sentiment analysis. Starting from the preparation of the development environment, the project progressed through the integration of IBM Watson NLP services, the implementation of a dedicated sentiment analysis module, the packaging of the application into a reusable Python package, and the validation of its functionality through unit testing. The application was subsequently deployed using the Flask web framework, enabling sentiment analysis capabilities to be accessed through a user-friendly web interface. Finally, static code analysis was performed to verify compliance with Python coding standards and assess overall code quality.

From a technical perspective, the project provided practical experience with Natural Language Processing technologies, REST-based service integration, web application development, software testing methodologies, and code quality assessment techniques. It also illustrated how pre-trained AI models can be incorporated into software systems without requiring the development and training of machine learning models from scratch.

Overall, the project successfully achieved its educational objectives by combining Artificial Intelligence and software engineering practices within a single application. Through the implementation of this sentiment analysis system, valuable experience was gained in the design, testing, deployment, and maintenance of modern AI-enabled applications.

References

1. <https://www.coursera.org/professional-certificates/ibm-generative-ai-engineering>, “IBM Generative AI Engineering”, Coursera.
2. <https://www.ibm.com/think/topics/sentiment-analysis>, “What is sentiment analysis?”, IBM.
3. <https://www.ibm.com/docs/en/watson-libraries>, “IBM Watson Natural Language Processing Library for Embed”, IBM.
4. <https://flask.palletsprojects.com/en/stable/>, “Flask’s documentation”, Pallets Projects.
5. <https://skills.network/lab-tools>, “Skills Network Labs”, IBM.
6. <https://github.com/ibm-developer-skills-network/zzrjt-practice-project-emb-ai>, “IBM Developer Skills Network. *AI-based Web Application Development and Deployment Project Repository*”.
7. <https://www.ibm.com/docs/en/watson-libraries?topic=catalog-sentiment-aggregated>, “IBM Watson Natural Language Processing Library for Embed Documentation”, IBM.
8. <https://martinfowler.com/bliki/UnitTest.html>, “Unit Test”, Martin Fowler
9. <https://docs.python.org/3/library/unittest.html>, “unittest — Unit testing framework Documentation”, Python Software Foundation